

From a vague complaint to a **verified fix**, in one tap.

A grounded troubleshooting engine for Samsung PRISM Theme 02: every step traced to its source, every Settings fix one exact deeplink away, every repeat answered from cache in milliseconds.



Evidence-locked
steps



Exact-screen
deeplinks



Relevance
gate



Contrastive
cache



One vague sentence costs an agent ~15 minutes - and the customer still hunts through Settings.

Theme 02 asks for a REST engine that turns such complaints into a grounded, ordered, one-tap plan under hard latency, cost and hygiene rules.

THE INPUT

"My Nexa X1 screen inputs are delayed and the touch responsiveness is laggy, causing a noticeable delay when I try to interact with the phone."

input.txt, line 19 + SLS 'Touchscreen issues' (9 sections)

no setting names

no technical terms

no order

1.3 s

THE REQUIRED OUTPUT (SCHEMA.PY CONTRACT)

goal	Follow these steps to perform this Touch Lag Troubleshooting
title	Touch lag recovery
— actionName	Configure Navigation Bar
— description	It will stop gestures blocking your taps
— category	auto
— steps	Tap Display → Tap Navigation bar → Select Buttons to turn off full scree...
— actionableDeepink	voiceassist://masked/act/2f3dd95259
— validationDeepink	key = 'Navigation bar'

HARD CONSTRAINTS WE ENFORCE IN CODE (THEME 2 PDF §4 AND §6)



0 URL leaks

regex gate on every field



5–7-word descriptions

validators + repair



Critical steps last

disruption-cost ordering



≤ 300 ms cached

P95, no LLM call



≤ 8 s cold

P95, two parallel LLM calls

The provided data is adversarial - a textbook RAG pipeline walks into **six traps**.

We audited all 578 catalog rows and 20 SIIS articles before writing code (docs/DATA_AUDIT.md).

TEXTBOOK PIPELINE

LLM writes the steps



Top-1 embedding match



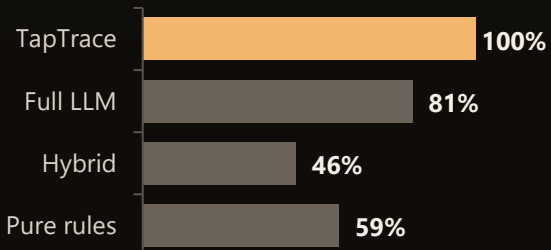
One global cache threshold



Trust any SIIS article



Exact deeplinks, 37-case benchmark



21

TV & appliance screens hidden in the phone catalog

DL-0468 ... DL-0569

→ Catalog Compiler quarantines them

14

labels that name the wrong feature

DL-0162 "Disable Grayscale" = mono audio

→ Match on screen labels, never messages

0.94

cosine of "won't turn ON" vs "won't turn OFF"

embeddings are polarity-blind

→ Facet guard + per-entry τ

4/20

SIIS articles are the wrong document

row_8: TV mirroring for a small phone screen

→ Relevance Gate → honest no_match

DL-0022

nearest entry to "factory data reset"

it is the AUTO reset after failed unlocks

→ Abstain → dummy_positive

12 words

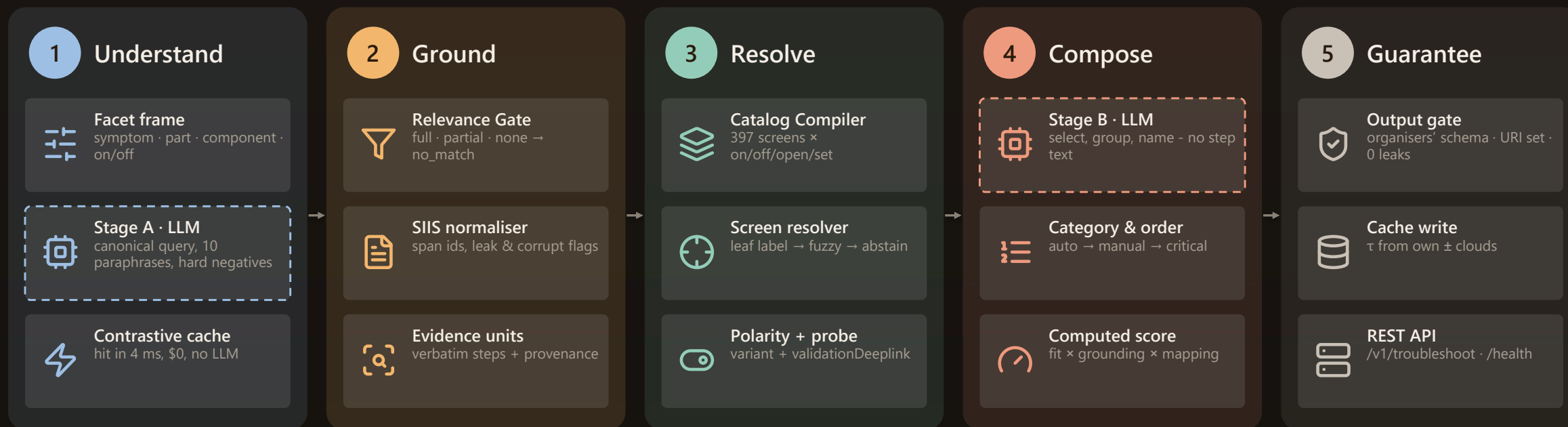
in the official sample description (5-7 allowed)

sample_output.json

→ Every field rule enforced in code

A five-stage pipeline where the LLM may select and name - but **never write** - a step.

Two LLM calls run in parallel on the cold path. Everything else is deterministic, tested code.



LATENCY BUDGET (MEASURED)

Cache hit

normalise + facets

embed (INT8 ONNX)

ANN + guard

P95 3.5 ms · \$0

Cold path

gate + units

Stage A || Stage B (LLM)

resolve + compose + gate

P50 1.3 s · P95 3.8 s



\$0.00013
per cold query



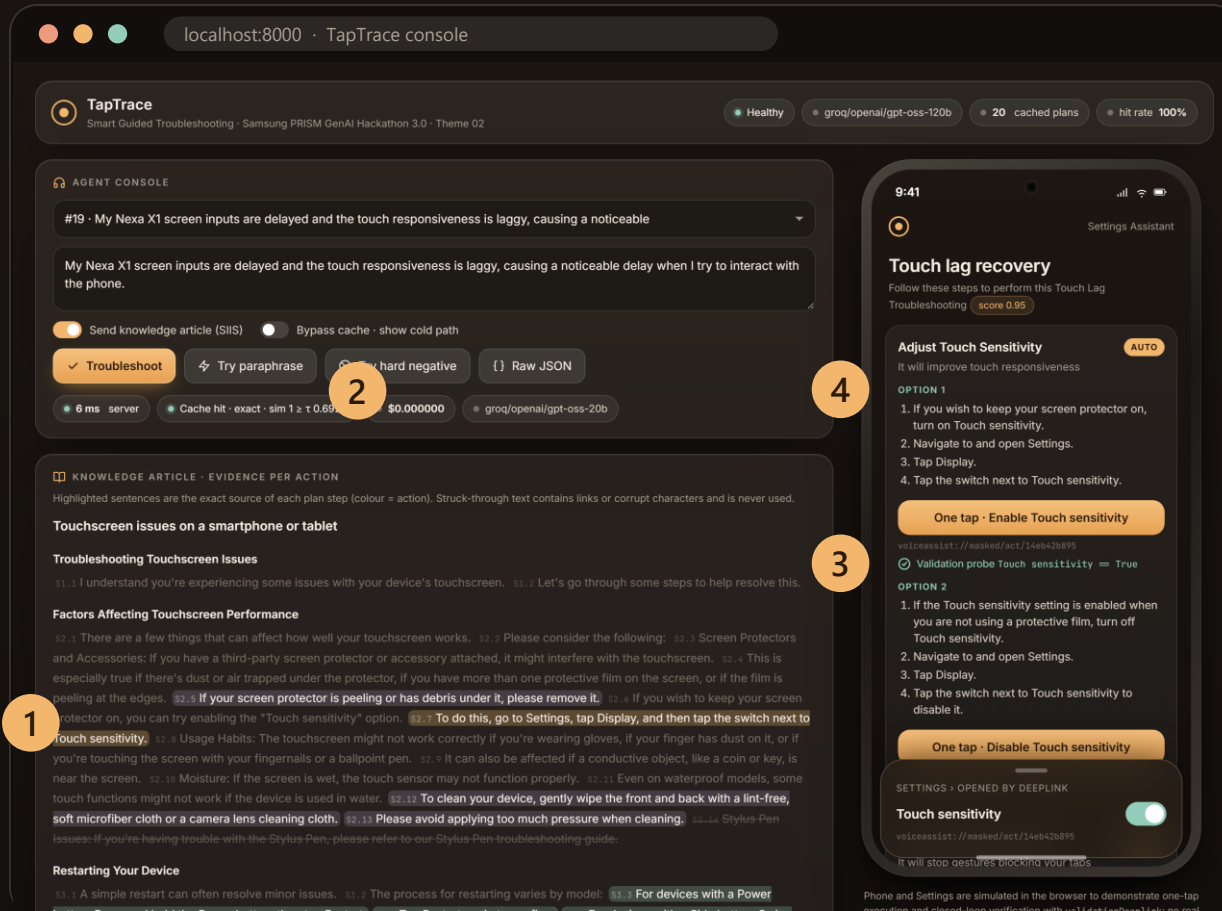
\$0
per cache hit



2
LLM calls per cold query

The live console shows every step's source sentence and **closes the loop** with a probe.

Real screenshot of demo/ served by the same container (device simulated in the browser).



1

Provenance

every plan step is highlighted in its source SIIS sentence, with span ids

2

Cache decision

exact or semantic hit with similarity $\geq$ per-entry τ at \$0, or the guard that refused

3

One tap + probe

an auto step runs its deeplink; the validationDeeplink probe turns ✓

4

Conditional options

one screen, two step groups: ON with a screen protector, OFF without

WHAT THE PHONE RECEIVED (RESULTS.JSONL, LINE 19)

```
"actionName": "Adjust Touch Sensitivity", "category": "auto",
"deeplink": "voiceassist://masked/act/14eb42b895", "originalType": "onURL",
"validationDeeplink": {"key": "Touch sensitivity",
"condition": "equal", "value": "True"}
```

Small, **CPU-only** and reproducible - one container, no GPU, and it runs even without an API key.

Every dependency is pinned in requirements.txt; the INT8 embedder and pre-warmed cache ship in the image.

SERVING



FastAPI



Uvicorn



Pydantic



ORJSON

INTELLIGENCE



gpt-oss-120b



Groq API



MiniLM-L6



ONNX Runtime

DATA & QUALITY



Python 3.12



NumPy



SQLite



pytest

```
taptrace/ catalog.py · siis.py · facets.py · resolver.py · plan.py · cache.py · engine.py · api.py
scripts/ build_results · evaluate · stress_test · audit_report · build_deck
eval/    gold · mapping_benchmark · heldout + test paraphrases · hard_negatives
Dockerfile · docker-compose.yml · results.jsonl · metrics.md · tests/ (27 gates)
```


23 MB

INT8 ONNX embedder, 0.97 cosine vs FP32


1.6 s

container cold start to healthy


495 req/s

16 concurrent clients, 0 errors


27 gates

pytest: contract, traps, gate, cache, API, unseen domains

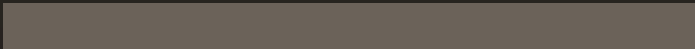

0 keys

needed to run: deterministic path + cache

From ~15 minutes of manual triage to ~1 seconds - and \$0 for every repeat complaint.

About 674× faster on a new complaint; a cache hit is served in 2 ms.

TIME TO AN ACTIONABLE PLAN (TO SCALE)

Manual triage  ≈ 15 min

TapTrace, new  1.3 s

TapTrace, repeat  2 ms

New-complaint time is the measured cold-path P50; repeat time is the measured cache-hit P50.

AT PRODUCTION SCALE: 10,000 SCENARIOS (BROCHURE TARGET)

Manual triage 2,500 agent-hours
at 15 min each

TapTrace first pass ≈ \$1.30 LLM cost
3.7 h sequential; parallelisable

Every repeat after \$0 · milliseconds
served from the contrastive cache

Mapping reuse shared step-path memo
screens resolved once, reused everywhere



Customer

BEFORE Reads text steps, then hunts nested menus.

AFTER Taps once: the exact screen and toggle state opens; a probe confirms the fix.



Support agent

BEFORE Reads, selects and orders steps by hand.

AFTER Gets an ordered plan with every step highlighted in its source article.



Operations

BEFORE Wrong articles turn into wrong advice.

AFTER Wrong-document tickets return no_match; ≈\$0.0001 per new scenario.

Every target in the brief is met - measured on the provided data and a frozen test set.

All numbers are produced by scripts/evaluate.py and artifacts/stress.json; nothing is typed by hand.

**100%**

schema-valid
target $\geq 99\%$

**0**

URL leaks
target 0

**3.00/3**

step accuracy
vs hand-annotated gold

**100%**

exact deeplinks
37 cases, 5 domains

**80%**

paraphrase hits
frozen test, target $\geq 80\%$

**3.8 s**

cold-path P95
target ≤ 8 s

INNOVATION HIGHLIGHTS



Evidence-locked steps the LLM can never author a step



Compiled catalog 397 screens × on / off / open / set



Relevance Gate honest no_match on wrong articles



Contrastive cache per-entry τ + symbolic facet guard

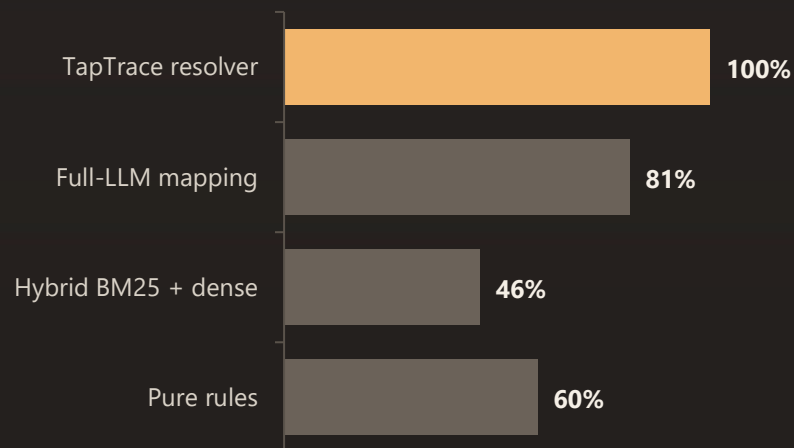


Conditional polarity one screen, two step groups



Closed loop verbatim validation probes

EXACT DEEPLINKS · SAME 37 INPUTS



LIMITATIONS (HONEST)



Free-tier LLM: ~3 new scenarios per minute; overflow degrades to deterministic wording, never fails



22% of unseen rewordings still miss or mis-hit (frozen test)



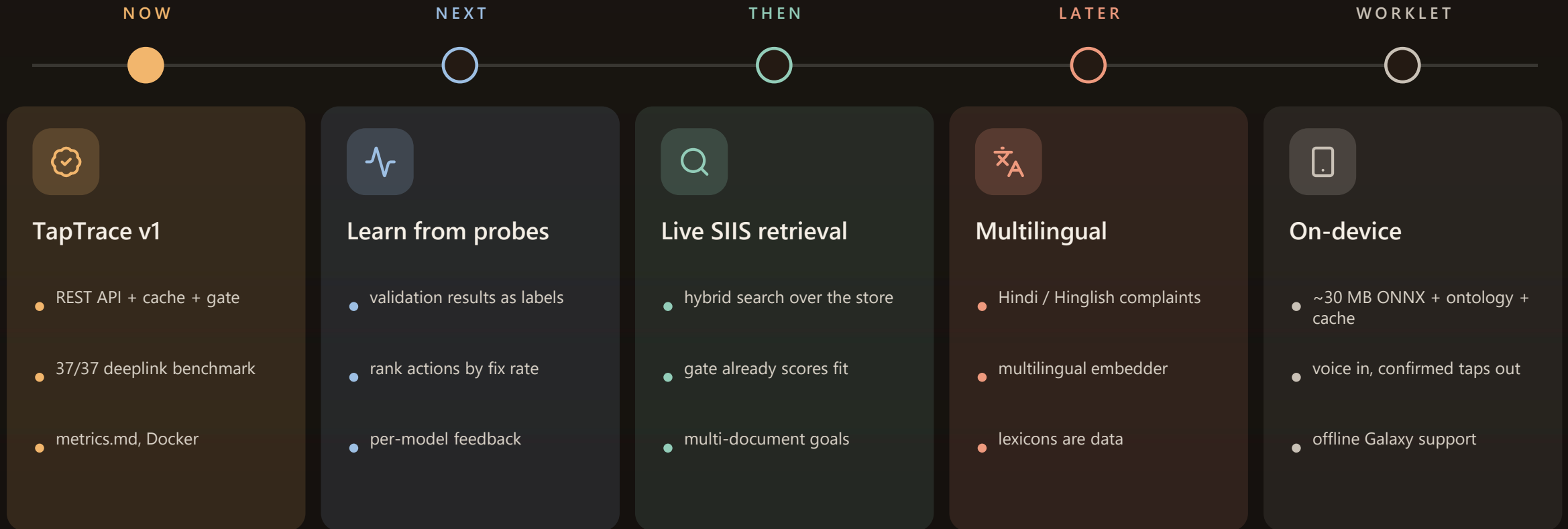
Facet lexicons cover six domains; new vocabulary relies on embeddings alone



Validation probes shown on a simulated device

From hackathon engine to Galaxy support worklet - every next step **reuses what is built.**

Each phase extends an existing component rather than adding a new system.



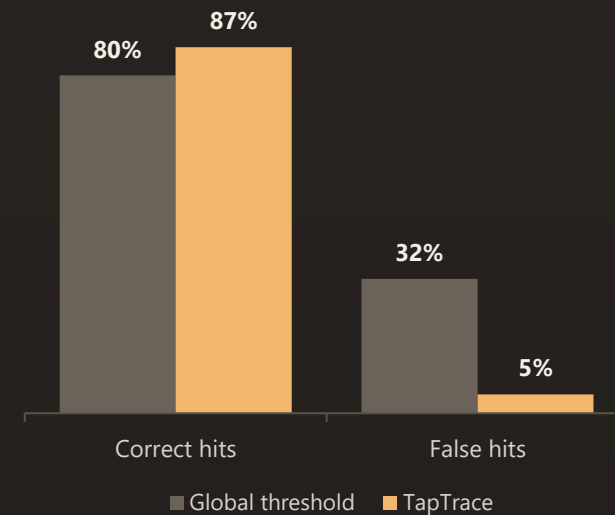
Why it can become a worklet: the data audit, relevance gate and compiled ontology transfer to any catalog, and every decision is traceable for review.

Where TapTrace differs from a typical pipeline - with the evidence for each.

Each row points at code, data or a measurement in the repository.

CAPABILITY	TYPICAL	TAPTRACE	EVIDENCE
Hallucinated steps	⊗ possible	✓ impossible by construction	taptrace/siis.py
Wrong-document article	⊗ extracts anyway	✓ no_match on 4/20, all as gold	eval/gold.json
On / off twin deeplinks	⊗ coin flip	✓ polarity picks the variant	resolver.py
Near-miss reset screen	⊗ DL-0022 (wrong)	✓ abstains to dummy_positive	M20 in benchmark
Cache false hits	⊗ 32% (dev)	✓ 5% (dev)	metrics.md §5
validationDeeplink	⊗ left null	✓ verbatim probe + demo ✓	results.json1
Runs without API key	⊗ no	✓ yes - deterministic + cache	TAPTRACE_OFFLINE=1

SEMANTIC CACHE · DEV SET



Frozen test (never tuned on): 80% hits, 77.5% correct, 6.7% false hits.

Everything the submission guideline asks for is in the **tagged commit**.

Release tag
PRISM_GENAI_HACKATHON_Y2026 marks
the judged commit.

	Working prototype code - public or shared GitHub repo	github.com/Arnav020/Tap-Trace · live: taptrace.onrender.com	YES
	README with reproducible setup: Docker, compose, venv, tests, evaluation	README.md	YES
	Demo video, max 5 minutes (YouTube or Drive)	https://youtu.be/72OtbD-1QvM	YES
	Presentation file named CollegeName_TeamName	submission/TIET_TapTrace.pptx + .pdf	YES
	Release tag PRISM_GENAI_HACKATHON_Y2026 on the final commit	everything referenced is in the tagged commit	YES
	results.jsonl · metrics.md · AI disclosure form · docs	repository root · submission/ · docs/	YES



Thank you.

Grounded steps. Exact screens. Honest answers when the knowledge does not fit.

100%

exact deeplinks

80%

paraphrase cache hits

0

URL leaks

4/20

wrong articles refused

ARNAV JOSHI · ajoshi4_be23@thapar.edu · THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY · github.com/Arnav020/Tap-Trace · LIVE taptrace.onrender.com

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India